

PREPRINT
OCTOBER 2026

A Machine-Checked Semantics of LLM Inference

Sampling, Attention, Tokenization, Chat, Decoding, and Diffusion in Lean 4

Daniel Arango Sohm

Universidad EAFIT, Medellín, Colombia
Former member of the QPT team

ABSTRACT

The inference side of a large language model is a stack of small algorithms: tokenizers, attention kernels, key–value caches, sampling filters, speculative decoders, chat templates, and diffusion samplers. Each has several independent implementations, and those implementations are usually tested against one another rather than against a definition. We give these components a single mathematical semantics, formalized in Lean 4 against Mathlib. The development comprises about 4,600 lines and 270 named theorems, and every declaration depends only on Lean's three standard axioms. It proves the exact algebra of sampling filters, the losslessness and optimality of speculative sampling, and the correctness of key–value caching and of FlashAttention-style online softmax as a commutative monoid. It proves that byte-pair encoding is lossless under every merge priority, together with a decidable criterion for when the Hugging Face and `tiktoken` conventions agree, a protocol checker for tool calls, and the time-independence of the masked-diffusion denoiser. It also proves six negative results. No sampling rule that treats tokens symmetrically can always keep exactly k tokens. No tokenizer that compresses can respect concatenation. Every chat template that writes its role markers in the content alphabet is ambiguous. No transformer without positional information can copy a fixed position. An autoregressive model need not define a distribution over strings. Parallel unmasking in diffusion is inexact. From the same definitions we build an executable oracle that evaluates test cases in exact rational arithmetic, and a harness that measures implementations against it. On 2,400 random merge tables, Hugging Face `tokenizers` and `tiktoken` each agree with the oracle on all 13,779 test words, while disagreeing with each other on 3.2% of them. On 2,400 random sampler chains, every divergence of llama.cpp from the tie-keeping semantics is accounted for exactly by the positional semantics that the impossibility theorem forces.

KEYWORDS Lean 4 · Mathlib · formal verification · LLM inference · sampling · tokenization · attention · speculative decoding · diffusion language models

01 Introduction

A deployed language model is a function from parameters and a context to a distribution over tokens. Turning that distribution into text requires more machinery. A tokenizer maps strings to token sequences. Attention kernels compute the model's reads over a cached context. A chain of sampling filters reshapes the next-token distribution. A speculative decoder may verify drafted tokens, a chat template linearizes a conversation, and a diffusion sampler may unmask several positions at once. Each of these components exists in many implementations, such as Hugging Face `transformers` and `tokenizers` [30], `tiktoken`, vLLM [13], and llama.cpp. Their tests mostly compare one implementation with another, or with remembered behavior.

This paper follows a different discipline, *denotational design* [6]. One first says precisely what a thing means, then requires every operation to preserve that meaning, and lets the implementation be whatever the equations force it to be. The approach is that of `ledger-semantics` [14], which formalizes double-entry accounting in Lean and uses the result as a test oracle for the C++ Ledger program. We apply it to LLM inference. A sampling filter means conditioning a distribution on a set. A KV-cached decoder means causal attention over the whole prefix. A FlashAttention kernel means a softmax-weighted average. A chat template means an injective encoding of a transcript. The theorems state when an implementation preserves that meaning, and several of them state that no implementation can.

Contributions.

- A Lean 4 formalization against Mathlib [18, 4] of sampling, speculative decoding, softmax and attention, tokenization, chat transcripts, autoregressive decoding, and masked diffusion (Sections 3 to 10). All definitions are stated over an arbitrary linearly ordered field, so one proof covers the real numbers of the literature and the exact rationals of the oracle.
- Positive results in a common framework. They include the algebra of sampling filters, the optimality of speculative sampling, an online-softmax commutative monoid, a generic correctness theorem for BPE with a decidable agreement criterion between the two conventions in use, a protocol checker for tool calls, and the any-order chain rule for masked diffusion.
- Six negative results that explain design constraints visible across implementations (Section 11).
- An executable oracle that evaluates case files under the proved definitions in exact arithmetic, and a measurement of Hugging Face `tokenizers`, `tiktoken`, and llama.cpp against it (Sections 12 and 13).

Several of the positive results are known: losslessness of speculative sampling [15, 2], correctness of online softmax [19, 3], time-independence of the absorbing-diffusion posterior [22], and the role of permutation equivariance [32]. Our contribution for these is a machine-checked statement in one framework, connected to an oracle. To our knowledge the following are not stated elsewhere in the form given here: the impossibility of symmetric exact top-k and nucleus selection, the no-compression theorem for homomorphic tokenizers, the ambiguity of every in-band chat template, and the decidable agreement criterion between merge-rule and concatenation-rank BPE.

TABLE 1 Modules of the development. Lines and named theorems and lemmas per module.

MODULE	CONTENT	LINES	THMS
`Dist`	finite distributions, conditioning, total variation, FinStoch	383	34
`Sampling`	top-k, top-p, min-p, temperature, ties, inverse CDF	732	46
`Counterexamples`	kernel-checked failures of sampler laws	52	4
`Speculative`	speculative sampling, couplings	194	17
`Softmax`	exponential characters, masking, temperature	220	23
`Attention`	equivariance, causality, KV cache, online softmax	573	38
`Real`	exp as a character; textbook restatements	51	4
`Tokenization`	free-monoid decoding, BPE conventions, enumeration	860	49
`Chat`	tool-call protocol, rendering	440	21
`Decoding`	mass conservation, tightness, greedy decoding	245	15
`Diffusion`	masked diffusion, posterior, chain rule	379	18
`Oracle`, `Driver`	case parsing and exact evaluation	465	1
Total		4,594	270

02 Method and trust base

The development is a Lean 4 library (Lean 4.34.1, Mathlib v4.34.1) of eleven theory modules and two executable modules (Table 1). It is built with warnings promoted to errors, so an unproved theorem (`sorry`) fails the build. A separate audit program walks every declaration in the library, 853 in all, and fails unless each depends only on propositional extensionality, the axiom of choice, and quotient soundness. The audit has been observed to fail: a planted proof by `native_decide`, which introduces an auxiliary axiom trusting compiled code, is rejected. The warnings-as-errors build alone does not catch it. Concrete counterexamples are therefore proved by `decide`, whose certificate the kernel checks by evaluation.

Numbers. Every probability lives in an arbitrary field $\mathbb{k}$ with a compatible linear order (`[Field K] [LinearOrder K] [IsStrictOrderedRing K]`). The oracle instantiates $\mathbb{k} = \mathbb{Q}$, so its arithmetic is exact. The analytic statements instantiate $\mathbb{k} = \mathbb{R}$. No measure theory is used. Sampling is made precise through interval statements (Theorem 8), and tightness through masses at finite depth (Section 9).

03 Foundations

DEFINITION 1 Finite distribution

For a finite type α, $\mathrm{Dist}_{\mathbb{k}}(\alpha)$ is the type of functions $p : \alpha \to \mathbb{k}$ with $p(a) \geq 0$ for all a and $\sum_a p(a) = 1$.

LEAN `Dist`

```
structure Dist (𝕜 : Type u) [Field 𝕜] [LinearOrder 𝕜] [IsStrictOrderedRing 𝕜]
    (α : Type v) [Fintype α] where
  prob : α → 𝕜
  nonneg : ∀ a, 0 ≤ prob a
  sum_eq_one : ∑ a, prob a = 1
```

The single operation through which masks, filters and grammar constraints act is conditioning. For $S \subseteq \alpha$ with $p(S) > 0$, write $p|_S(a) = [\![a \in S]\!]\, p(a)/p(S)$ (`Dist.restrict`).

LEMMA 2 **Conditioning**
$(p|_S)|_T = p|_{S\cap T}$ whenever the masses involved are positive. Moreover $p|_S = p|_{S'}$ whenever S and S' contain the same outcomes of positive probability.

LEAN `restrict_restrict, restrict_congr`

The finite distribution monad (point masses and `bind`) satisfies the monad laws. Finite types and stochastic maps $\alpha \to \mathrm{Dist}_{\mathbb{k}}(\beta)$ form a category $\mathrm{FinStoch}_{\mathbb{k}}$, given as an instance of Mathlib's `Category` whose composition is Kleisli composition (`FinStoch.category`). Total variation is $\mathrm{tv}(p,q) = \sum_a \max(0, p(a) - q(a))$. We prove $\mathrm{tv}(p,q) = \frac{1}{2}\sum_a |p(a) - q(a)|$, $\sum_a \min(p(a), q(a)) = 1 - \mathrm{tv}(p,q)$, and $\mathrm{tv}(p,q) = 0 \iff p = q$.

04 Sampling

Let V be a finite vocabulary and $p : V \to \mathbb{k}$. Write $\mathrm{above}_p(v) = \{u : p(v) < p(u)\}$ for the tokens strictly preferred to v. The three standard filters keep the sets

$$\mathrm{TopK}_k(p) = \{v : |\mathrm{above}_p(v)| < k\}, \qquad \mathrm{MinP}_m(p) = \{v : \forall u,\ m\,p(u) \le p(v)\},$$
$$\mathrm{TopP}_P(p) = \left\{v : \textstyle\sum_{u \in \mathrm{above}_p(v)} p(u) < P\right\},$$

and the filtered distribution is the conditioning $p|_S$ on the kept set S. For distinct probabilities, TopP_P is the usual rule "sort in descending order and keep the shortest prefix whose mass reaches P". With ties, all three keep every tied token. Temperature $1/n$ is $\mathrm{sharpen}_n(p)(v) = p(v)^n / \sum_u p(u)^n$, which stays inside $\mathbb{Q}$. In Section 6 we show that it equals $\mathrm{softmax}(n \cdot z)$ when $p = \mathrm{softmax}(z)$. Every kept set contains a most probable token, provided $k \ge 1$, $P > 0$ and $m \le 1$, so no filter can produce an empty distribution.

THEOREM 3 **Sampler algebra**
For $k, k' \ge 1$, $m, m' \le 1$ and $n \ge 1$, and writing filters in the order they are applied,

(i) top-k then top-k' equals top-$\min(k, k')$;

(ii) min-m then min-m' equals min-$\max(m, m')$; in particular both filters are idempotent;

(iii) top-k and min-m commute;

(iv) top-k and temperature $1/n$ commute.

LEAN `topK_topK, minP_minP, topK_minP_comm, topK_sharpen_comm`

PROOF SKETCH All three kept sets are *upward closed*: if v is kept and $p(v) < p(u)$, then u is kept. Conditioning on an upward-closed set S leaves $\mathrm{above}(v)$ unchanged for $v \in S$

(`above_restrict`), and min-p is insensitive to conditioning on a set that contains a maximizer (`minP_restrict_iff`). Each identity then reduces, through Lemma 2, to an equality of kept sets. For (iv), $x \mapsto x^n$ is strictly increasing on $[0, \infty)$, and top-k depends only on the order of the scores (`topKSet_congr`). ∎

The laws one might expect beyond Theorem 3 are false. With $p_0 = (\frac{1}{2}, \frac{3}{10}, \frac{1}{5})$, the kernel evaluates the following (`Counterexamples`).

PROPOSITION 4 Failed laws

(i) Top-p is not idempotent: one pass of $\mathrm{TopP}_{3/5}$ gives token 1 the probability $\frac{3}{8}$, and a second pass gives it 0.

(ii) Top-p does not commute with top-k: top-p then top-2 leaves token 1 at $\frac{3}{8}$, while top-2 then top-p removes it.

(iii) Top-p does not commute with temperature $\frac{1}{2}$: token 1 ends at $\frac{9}{34}$ in one order and at 0 in the other.

(iv) Min-p ($m = \frac{1}{2}$) does not commute with temperature $\frac{1}{2}$, with the same numbers.

A sampler chain may therefore reorder top-k, min-p and temperature only as Theorem 3 allows, and may not move top-p at all.

Ties. Implementations differ on ties, and the difference is forced. Call a selection rule $\mathrm{sel} : (V \to \mathbb{k}) \to \mathcal{P}(V)$ *equivariant* if $\mathrm{sel}(p \circ \sigma^{-1}) = \sigma[\mathrm{sel}(p)]$ for every permutation σ of V, that is, if relabeling the tokens relabels the output.

THEOREM 5 No symmetric exact top-k

Let $0 < k < |V|$. No equivariant selection rule satisfies $|\mathrm{sel}(p)| = k$ for every distribution p.

LEAN `no_equivariant_exact_topK`

PROOF On the uniform distribution u we have $u \circ \sigma^{-1} = u$ for every σ, so $\mathrm{sel}(u)$ is invariant under all permutations. If it contains some v, then for any w the transposition $(v\,w)$ maps v to w, so it contains w too. Hence $\mathrm{sel}(u)$ is either empty or all of V, and neither has k elements. ∎

```
theorem no_equivariant_exact_topK [DecidableEq V] {k : ℕ}
    (hk0 : 0 < k) (hkn : k < Fintype.card V)
    (sel : (V → 𝕜) → Finset V) (hsel : Equivariant sel) :
    ¬ ∀ d : Dist 𝕜 V, (sel d.prob).card = k
```

THEOREM 6 No symmetric smallest nucleus

Let V be nonempty and $0 < P \le 1 - 1/|V|$. No equivariant rule returns, for every distribution, a set of minimum cardinality among those of mass at least P.

LEAN `no_equivariant_exact_nucleus`

The tie-keeping filters are equivariant (`topKSet_equivariant`). The other option is to break ties by position. Define $u \prec_p v$ if $p(u) > p(v)$, or if $p(u) = p(v)$ and $u < v$, and let the positional filters count predecessors under $\prec_p$ instead of above_p (`Dist.topKPos`, `Dist.topPPos`).

THEOREM 7 Positional tie-breaking

Positional top-k keeps exactly $\min(k, |V|)$ tokens. If p is injective, the positional and tie-keeping filters coincide. For $0 < k < |V|$, positional top-k is not equivariant.

LEAN `card_topKPosSet, topKPosSet_eq_of_injective, topPPosSet_eq_of_injective, topKPosSet_not_equivariant`

An implementation must therefore choose between keeping exactly k tokens and treating tokens symmetrically. Hugging Face `transformers` (read from the source of version 5.19.0) keeps ties in top-k, since it removes only scores strictly below the k-th largest, but breaks them by sort position in top-p. llama.cpp breaks ties by position in both, as measured in Section 13.

THEOREM 8 Inverse-CDF sampling
Let $F(i) = \sum_{j<i} p(j)$ on $V = \{0, \dots, n-1\}$ and let $\iota(u)$ be the least i with $u < F(i+1)$. For $0 \le u < 1$, $\iota(u) = i \iff F(i) \le u < F(i+1)$, and $F(i+1) - F(i) = p(i)$. A uniform u therefore selects i with probability $p(i)$.

LEAN `invCdf_eq_iff, cdf_interval_length`

05 Speculative decoding

A draft model proposes $x \sim q$. The target, with distribution p, accepts with probability $a(x) = \min(1, p(x)/q(x))$, and on rejection a token is drawn from the residual $r = (p - q)^+ / \mathrm{tv}(p, q)$ [15, 2]. We define the joint law of (draft, emitted token) exactly as the algorithm runs,

$$\pi(x, y) = q(x)\big(a(x)\,[\![y = x]\!] + (1 - a(x))\,r(y)\big).$$

THEOREM 9 Speculative sampling

(i) The draft has law q: $\sum_y \pi(x, y) = q(x)$.

(ii) *Losslessness*: the emitted token has law p: $\sum_x \pi(x, y) = p(y)$.

(iii) The acceptance probability is $\sum_x q(x)a(x) = 1 - \mathrm{tv}(p, q)$, and the emitted token equals the draft with the same probability.

(iv) *Optimality*: every coupling γ of q and p has $\sum_x \gamma(x, x) \le 1 - \mathrm{tv}(p, q)$.

LEAN `specJoint_fst, specJoint_snd, accept_rate, specJoint_diag, coupling_le, spec_optimal`

PROOF SKETCH Since $q(x)a(x) = \min(p(x), q(x))$, the total rejection probability is $\mathrm{tv}(p, q)$, and a rejected draft is never re-emitted, because $r(x) = 0$ whenever $q(x) > p(x)$. For (iv), $\gamma(x, x) \le \min(q(x), p(x))$ by the marginal constraints. This is the classical coupling inequality [16]. ∎

Because any lossless rule for verifying one drafted token is a coupling, (iv) shows that speculative sampling has the highest acceptance rate any such rule can achieve.

06 Softmax and attention

DEFINITION 10 Exponential character
An *exponential character* from an ordered additive group G of scores into $\mathbb{k}$ is a strictly increasing

map $E : G \to \Bbbk$ with $E(a) > 0$ and $E(a+b) = E(a)E(b)$. For $z : \iota \to G$, $\mathrm{softmax}_E(z)_i = E(z_i)/\sum_j E(z_j)$.

LEAN ExpChar

exp : $\mathbb{R} \to \mathbb{R}$ is a character (`ExpChar.real`). So is $z \mapsto 2^z : \mathbb{Z} \to \mathbb{Q}$ (`ExpChar.base2`), which the oracle uses to evaluate softmax with no rounding. Every result in this section holds for every character, and so for both.

THEOREM 11 Softmax

(i) $\mathrm{softmax}_E(z + c) = \mathrm{softmax}_E(z)$; this licenses subtracting the maximum logit.

(ii) $\mathrm{softmax}_E(z)_i < \mathrm{softmax}_E(z)_j \iff z_i < z_j$.

(iii) *Masking is conditioning*: setting logits outside S to $-\infty$ and taking softmax gives $\mathrm{softmax}_E(z)|_S$.

(iv) For G a field and $T > 0$, $\mathrm{TopK}_k(\mathrm{softmax}_E(z/T)) = \mathrm{TopK}_k(z)$.

(v) $\mathrm{softmax}_E(n \cdot z) = \mathrm{sharpen}_n(\mathrm{softmax}_E(z))$.

LEAN softmax_add_const, softmax_lt_iff, softmaxOn_eq_restrict, topK_temperature, softmax_nsmul

One read. $\mathrm{attend}(s, v) = \sum_j \mathrm{softmax}_E(s)_j \, v_j$ for scores s and values v_j in a $\Bbbk$-module.

THEOREM 12 Convexity

For every linear functional φ and constant c, if $\varphi(v_j) \le c$ for all j then $\varphi(\mathrm{attend}(s, v)) \le c$. If every $v_j = m$, then $\mathrm{attend}(s, v) = m$ for every s.

LEAN attend_convex, attend_const

The second statement says attention cannot abstain. Its weights always sum to one, which is the property that motivates attention-sink tokens [31].

Permutation equivariance. A map $f : (\iota \to X) \to (\iota \to Y)$ is equivariant if $f(x \circ \sigma) = f(x) \circ \sigma$ for every permutation σ.

THEOREM 13 Equivariance

Self-attention with any score function and no positional information is equivariant. So is any block $y = x + \sum_h \mathrm{attn}_h(\mathrm{norm}\, x)$, $\mathrm{out} = y + \mathrm{mlp}(\mathrm{norm}\, y)$, and any stack of such blocks.

LEAN selfAttn_equivariant, block_equivariant, stack_equivariant

THEOREM 14 Positional information is necessary

If ι has two distinct positions $i_0 \ne i_1$ and X has two distinct elements, no equivariant $f : (\iota \to X) \to (\iota \to X)$ satisfies $f(x)_i = x_{i_0}$ for all x and i.

LEAN no_equivariant_first

PROOF Let $x_{i_0} = a$ and $x_i = b$ elsewhere, and let $\sigma = (i_0\, i_1)$. Then $f(x \circ \sigma)_{i_0} = (x \circ \sigma)_{i_0} = b$, while equivariance gives $f(x \circ \sigma)_{i_0} = f(x)_{i_1} = x_{i_0} = a$. ∎

Without positional encodings, then, no transformer can output the token at a fixed position [32].

Causality and the KV cache. Causal attention over a list maps position i to a read with the query of x_i over the keys and values of $x_0, \dots, x_i$ (`causalAttn`). The cached decoder `kvDecode` appends one (key, value) pair per step and reads the cache, recomputing nothing.

THEOREM 15 **Causality and KV caching**
Causal attention is causal: it preserves length, and its output on a prefix is the prefix of its output. Causal maps are closed under composition and residual sums. *KV caching is exact*: kvDecode$(xs) =$ causalAttn(xs). For any causal f, $f(xs +\!\!+ [x]) = f(xs) +\!\!+ (\text{last entry})$.

LEAN `causalAttn_causal, kvDecode_eq, causal_comp, causal_extend`

Online softmax as a monoid. FlashAttention [3], memory-efficient attention [24] and ring attention [17] reduce attention over blocks of keys using the online normalizer of Milakov and Gimelshein [19]. A partial state is a triple (m, ℓ, o) of running maximum, normalizer and accumulator, and two states merge by rescaling to the larger maximum:

$$(m_1, \ell_1, o_1) \otimes (m_2, \ell_2, o_2) = \big(m,\ \ell_1 E(m_1 - m) + \ell_2 E(m_2 - m),\ E(m_1 - m)\, o_1 + E(m_2 - m)\, o_2\big),$$

where $m = \max(m_1, m_2)$.

THEOREM 16 **Online softmax**
Adjoining an identity for the empty block, $\otimes$ is a commutative monoid. Let single$(s, v) = (s, 1, v)$ and fin$(m, \ell, o) = \ell^{-1} o$. For every list of (score, value) pairs, fin$\big(\prod_j$ single$(s_j, v_j)\big) = \big(\sum_j E(s_j)\big)^{-1} \sum_j E(s_j) v_j$. The same holds for every partition of the list into blocks and every order of the blocks. The running maximum of a nonempty product is the maximum score, so every exponent the kernel evaluates is ≤ 0.

LEAN `Online, finalize_ofList, finalize_blocks, ofList_perm, ofList_max`

The fold is the free-monoid homomorphism that `single` induces (`ofList_eq_lift`). Correctness of tiling is then a homomorphism property rather than a calculation.

07 Tokenization

A token t spells a string str(t) over an alphabet C. Decoding is the free-monoid homomorphism dec : $T^* \to C^*$ that str induces (`decodeHom`).

THEOREM 17 **No compression**
Suppose every token spells a nonempty string. If enc : $C^* \to T^*$ satisfies dec $\circ$ enc $=$ id and enc$(a\,b) =$ enc(a) enc(b), then $|\text{enc}(s)| = |s|$ for every s, and every emitted token spells a single character. Consequently, any tokenizer with $|\text{enc}(s)| < |s|$ for some s violates enc$(a\,b) =$ enc(a) enc(b) for some a, b.

LEAN `homomorphic_no_compression, compressing_not_homomorphic`

PROOF SKETCH enc$([c])$ is nonempty and decodes to $[c]$. Since each token decodes to at least one character, it consists of exactly one token. Induct on s using enc$(c\,s) =$ enc$([c])$ enc(s). ∎

Tokenization therefore lies outside any compositional core, and boundary artifacts such as the prompt-boundary problem are consequences of compression rather than defects of one implementation. Tokenizations are not unique (`decode_not_injective`). `tokenizations` enumerates every token sequence over a vocabulary that spells a given string, and it is proved

sound, complete and duplicate-free (`tokenizations_spec`, `tokenizations_nodup`). The probability of a string under a token-level model is the sum over this enumeration [29, 9].

BPE, generically. Byte-pair encoding [27, 25, 33] repeatedly merges the leftmost occurrence of the lowest-priority adjacent pair. We define it once, for an arbitrary priority $f : T \times T \to \text{Option}(\mathbb{N} \times T)$ that returns a rank and a merged token (`bpeWith`).

THEOREM 18 Generic BPE
Each step merges a pair of minimum rank, and every pair to its left has strictly larger rank. The output contains no mergeable pair. BPE is idempotent. If f is *sound*, meaning a merged token spells the concatenation of its parts, then BPE is lossless.

LEAN `stepWith_spec, bpeWith_normal, bpeWith_idem, bpeWith_decode`

Two priorities are in use. Hugging Face `tokenizers` and the GPT-2 encoder rank a pair by its merge *rule* (`rankOf`). `tiktoken` ranks a pair by the rank of the *token it would form* (`concatRank`).

THEOREM 19 Agreement of conventions
Let $\mathcal{T}$ be a set of tokens closed under merging. If two priorities agree on every pair from $\mathcal{T}$, their BPE outputs agree on every sequence over $\mathcal{T}$. For a finite vocabulary this condition is decidable. Without it the conventions differ: for the merges $(b, c), (a, b), (ab, c)$, the word abc encodes as $a \cdot bc$ under rule ranks and as abc under concatenation ranks.

LEAN `bpeWith_congr, hf_eq_tiktoken, hf_ne_tiktoken`

`tiktoken` also emits a word directly when the word is itself a vocabulary token, and Hugging Face does the same with `ignore_merges`. We record this convention as `lookupFirst`. It preserves losslessness (`tikEncode_flatten`) and changes results (`lookup_matters`). Finally, BPE does not find the shortest tokenization (`bpe_not_shortest`).

08 Chat transcripts

A transcript is a list of events: system, user and assistant messages, call_i (a tool call with identifier i), and result_i. Let $c_i(\textit{pre})$ and $r_i(\textit{pre})$ count the calls and results with identifier i, and let $\text{NoPending}(\textit{pre})$ mean $r_i = c_i$ for all i. The protocol is stated per event, against the events before it.

$$\text{Allowed}(\textit{pre}, e) = \begin{cases} c_i(\textit{pre}) = 0 \wedge (\text{NoPending}(\textit{pre}) \vee \text{lastIsCall}(\textit{pre})) & e = \text{call}_i \\ c_i(\textit{pre}) = 1 \wedge r_i(\textit{pre}) = 0 & e = \text{result}_i \\ \text{NoPending}(\textit{pre}) & \text{otherwise.} \end{cases}$$

A transcript es is *valid* if every es_k is allowed after $es_{<k}$.

THEOREM 20 Protocol checker
A single left-to-right pass with a state of (called, answered, last-was-call) accepts exactly the valid transcripts. In a valid transcript every identifier is called at most once and answered at most as often as it is called, and every result is preceded by its call.

LEAN `check_iff_valid, valid_counts, result_has_call`

Rendering with out-of-band markers, special symbols disjoint from content, is invertible (`parse_render`), and stays injective through any injective tokenizer

(`render_encode_injective`). An *in-band* template writes each event as open(h) *body* close with open(h), close $\in C^*$.

THEOREM 21 In-band ambiguity
For every choice of open and close, in-band rendering is not injective.

LEAN `inBand_not_injective`

PROOF The single user message with body close · open(assistant) and the pair (empty user message, empty assistant message) both render to open(user) close open(assistant) close. ∎

Role separation exists only at the token level, and only while user text cannot produce a special token. This is the formal content behind the use of reserved tokens as a defense against prompt injection [10].

09 Autoregressive decoding

A model assigns each prefix a distribution $m(\mathit{pre}) \in \mathrm{Dist}_{\mathbb{k}}(V \cup \{\mathrm{eos}\})$. Let alive$_n$ be the probability of having generated n tokens without stopping, and stopped$_n$ the probability of having stopped within n.

THEOREM 22 Mass conservation and tightness
stopped$_n$ + alive$_n$ $= 1$ for every n and every starting prefix. If every prefix stops with probability at least ε, then alive$_n$ $\le (1-\varepsilon)^n$.

LEAN `mass_conservation, alive_le_of_stop`

THEOREM 23 A non-tight model
For the model that stops after L tokens with probability $1/(L+2)^2$, alive$_n$ $= \frac{n+2}{2(n+1)}$, so stopped$_n$ $< \frac{1}{2}$ for every n.

LEAN `leaky_aliveMass, stopped_lt_half`

This model never places half of its probability on finite strings, so an autoregressive model need not define a distribution over strings, the phenomenon studied as tightness by Du et al. [5].

THEOREM 24 Greedy is not MAP
There is a two-step model whose strictly most probable first token begins only sequences of probability $\frac{3}{10}$, while another sequence has probability $\frac{2}{5}$.

LEAN `greedy_not_map`

Greedy decoding therefore does not find the most probable sequence, under any tie-breaking.

10 Masked diffusion

Masked (absorbing) diffusion [1, 26, 28, 21] replaces each token, independently, with a mask symbol $\perp$. The forward kernel at keep rate α sends v to v with probability α and to $\perp$ with probability $1-\alpha$, and fixes $\perp$.

THEOREM 25 Forward process

$\alpha \mapsto \mathsf{mask}_\alpha$ is a monoid homomorphism from the multiplicative interval $[0,1]$ into the endomorphisms of $V \cup \{\bot\}$ in $\mathsf{FinStoch}_{\mathbb{k}}$, given as a Mathlib `MonoidHom`. The sequence-level forward probability is $q_\alpha(y \mid x) = [\![x \models y]\!]\,(1-\alpha)^{\#_\bot(y)}\alpha^{n-\#_\bot(y)}$, where $x \models y$ means y agrees with x wherever it is unmasked.

LEAN forward, maskProb_comp, qSeq_closed

THEOREM 26 Time-independent denoiser
For every data distribution p, every masked sequence y and every $0 < \alpha < 1$,

$$\frac{p(x)\,q_\alpha(y \mid x)}{\sum_{x'} p(x')\,q_\alpha(y \mid x')} = \frac{[\![x \models y]\!]\,p(x)}{\sum_{x'} [\![x' \models y]\!]\,p(x')}.$$

LEAN posterior_time_independent

The optimal denoiser is therefore the data distribution conditioned on the unmasked tokens, whatever the noise level, as observed by Ou et al. [22]. The factor that depends on α depends only on the mask pattern of y, and it cancels. The closed-form reverse step used by samplers is the Bayes posterior of the forward process (`reverse_bayes`).

THEOREM 27 Any-order chain rule
Let $o_1, \ldots, o_n$ enumerate the positions, and let c_k be the exact conditional probability that position o_k holds x_{o_k} given x on $o_1, \ldots, o_{k-1}$, with $0/0 = 0$. Then $p(x) = \prod_k c_k$, and c_k is the marginal of the posterior of Theorem 26 at o_k.

LEAN chain_rule_fin, posterior_marginal

Unmasking one position at a time with the exact denoiser therefore samples the data distribution exactly, in every order.

PROPOSITION 28 Parallel unmasking is inexact
For the distribution uniform on $\{(0,1),(1,0)\}$, the product of the two exact posterior marginals from the fully masked sequence gives $(1,1)$ the probability $\frac{1}{4}$, but $p(1,1) = 0$.

LEAN parallel_decoding_inexact

11 The negative results together

The six impossibility theorems share a shape. Each says that a property folklore expects of a component is inconsistent with that component's purpose, and so each explains a design constraint found in mature implementations.

(1) *Symmetric exact top-k does not exist* (Theorem 5). Every sampler must document its tie convention.

(2) *Compressing tokenizers are not compositional* (Theorem 17). Tokenization cannot be done piecewise, and prompt boundaries matter.

(3) *In-band chat templates are ambiguous* (Theorem 21). Roles require out-of-band special tokens.

(4) *Order requires positional information* (Theorem 14). Positional encodings are necessary, not an optimization.

```
@sample top-p is not idempotent
probs 1/2 3/10 1/5
top-p 3/5
emit
top-p 3/5
```

```
-- top-p is not idempotent
p|0|5/8   p|1|3/8   p|2|0
q|0|5/8   q|1|3/8   q|2|0
p|0|1     p|1|0     p|2|0
q|0|1     q|1|0     q|2|0
```

FIGURE 1 A case file and the oracle's output, with rows placed side by side here. Rows p use the tie-keeping filters and rows q the positional ones. Every number is an exact rational.

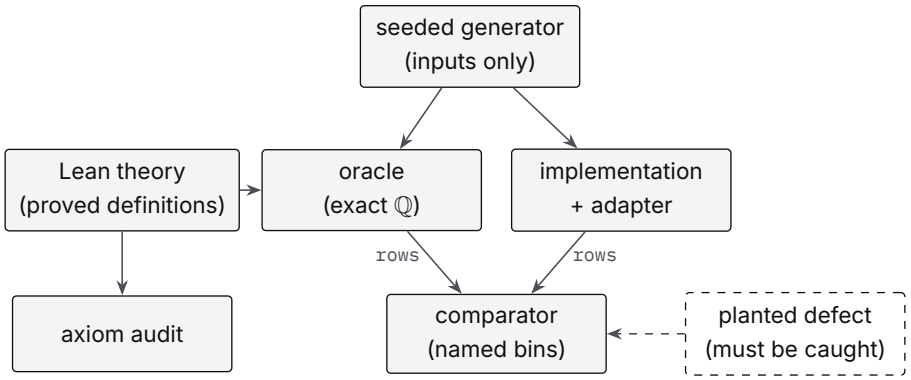


FIGURE 2 The measurement procedure. The generator never computes expected results; those come only from the oracle.

(5) *Autoregressive models need not be tight* (Theorem 23). Sequence-level probabilities need a stopping guarantee.

(6) *Parallel unmasking is inexact* (Section 10). Multi-token diffusion steps trade fidelity for speed.

12 The oracle

The oracle (`LLM/Oracle.lean`, `LLM/Driver.lean`) reads case files and prints rows computed by the proved definitions over ℚ. It runs under the Lean interpreter (`lake env lean --run LLM/Driver.lean FILE`), so what executes is a direct reading of the checked definitions. Inputs are validated at run time, for example that probabilities are nonnegative and sum to one. The checks are decidable propositions whose proofs construct the values the theorems speak about.

There are six case kinds. `@sample` runs a filter chain from probabilities, weights, or integer logits under the base-2 softmax, and prints both tie semantics (Fig. 1). `@speculative` prints the acceptance rate, total variation, per-token acceptance, residual, and emitted law. `@bpe` prints the tokens under rule ranks, under rule ranks with whole-word lookup, and under `tiktoken`'s convention, the number of tokenizations, and whether the hypotheses of Theorem 19 hold. `@chat` prints validity with the first offending event, and completeness. `@attention` prints softmax weights and outputs, also through the online-softmax monoid over blocks. `@diffusion` prints the evidence, posterior, and posterior marginals. A rejected case prints a single `!! ERROR` line.

TABLE 2 Measurements (`harness/run.sh 600`: four seeds of 600 cases each per corpus). Lean 4.34.1, Mathlib v4.34.1, `tokenizers` 0.23.3, `tiktoken` 0.14.0, llama.cpp commit a9d27ac (2026-10-11).

IMPLEMENTATION	COMPARED AGAINST	RESULT
HF `tokenizers`, BPE, defaults	rule-rank BPE (bpe)	13,779 / 13,779 words identical
HF `tokenizers`, `ignore_merges`	lookup, then rule-rank BPE	13,779 / 13,779 identical
`tiktoken`	lookup, then concatenation-rank BPE	13,779 / 13,779 identical
llama.cpp `top_k`, `top_p`, `min_p`, `temp`, no ties	tie-keeping filters	1,216 / 1,216 chains, $\lvert\Delta p\rvert \le 5.1 \times 10^{-8}$
llama.cpp, with ties	tie-keeping, else positional	939 + 245 / 1,184 chains; 0 unexplained
Planted defects (tokenizer, sampler)	—	both detected

13 Measuring implementations

The procedure follows `ledger-semantics` (Fig. 2). A seeded generator writes case files containing inputs only. It favors the inputs where implementations are known to have conventions: tied logits, words that are themselves tokens, and merge tables with several splits of one string. A small adapter per implementation runs the same cases and prints the oracle's row format. The comparator places every case in exactly one named bin (agree, diverge, skipped by either side, or missing), so nothing is dropped silently. Token rows are compared as strings. Sampler rows require that a zero be matched by a zero and that nonzero probabilities agree within 10^{-6}, since llama.cpp stores logits in `float32`. Each comparator is observed to fail: a planted defect, an adapter that glues two tokens or one that inverts temperature, must be detected or the run fails.

Tie divergences. An implementation that keeps exactly k tokens breaks ties in its own order, which need not be the oracle's order by index. Every filter treats tied tokens alike except for that choice, and tied tokens stay tied through temperature and renormalization. Such an implementation must therefore agree with the positional semantics up to a permutation of tied tokens. The harness checks exactly this: the multiset of (logit, probability) pairs over the kept tokens must equal that of the oracle's q rows. A divergence that fails this check is reported as unexplained.

Table 2 summarizes the results. Two findings came out of the comparisons. First, `tiktoken` initially disagreed with the oracle's concatenation-rank BPE on words that are themselves vocabulary tokens. The disagreement was a convention, whole-word lookup. Following the `ledger-semantics` rule that a divergence is either a defect in the implementation or a convention the semantics has not yet recorded, we recorded it as `lookupFirst`, with its own theorems, after which agreement is exact. Second, on the same random merge tables the two tokenizer libraries disagree with each other on 439 of 13,779 words (3.2%), and on 279 when Hugging Face uses `ignore_merges`. The oracle reproduces each library exactly, so every one of these disagreements is predicted. Theorem 19 states the condition under which they cannot occur.

For sampling, llama.cpp agrees with the tie-keeping semantics on every tie-free chain. On chains with ties, 245 of 1,184 diverge, and every one agrees with the positional semantics up to a permutation of tied tokens. Some of these divergences cascade: a tie cut by top-k changes the

renormalized masses that a later top-p sees. A check that compared kept sets only after the fact would misclassify such cases. Replaying the convention exactly classifies all of them.

14 Related work

Formal methods for machine learning. Most verification work on neural networks proves properties of trained models, for example with SMT-based methods [12]. We verify the model-independent algorithms around the model instead, and connect them to implementations through an oracle. The denotational method follows Elliott [6]. The use of a proved semantics as a test oracle, the procedure of planted defects, and the discipline of named skip bins follow `ledger-semantics` [14]. Our category of finite stochastic maps is a special case of Fritz's Markov categories [8].

Tokenization. Zouhar et al. give a formal analysis of BPE as an optimization problem [33]. Gastaldi et al. treat tokenizers as stochastic maps and study consistency of estimators [9]. Vieira et al. show how to turn token-level models into character-level ones [29], which requires exactly the enumeration of tokenizations we verify. Our no-compression theorem and the agreement criterion between BPE conventions complement this work at the level of exact identities.

Decoding. Top-k [7], nucleus [11] and min-p [20] sampling are usually specified by an algorithm over sorted arrays, which leaves ties unspecified. Theorem 5 shows that this gap cannot be closed by any symmetric rule. Speculative sampling [15, 2] is known to be lossless, and its acceptance rate is the maximal-coupling bound [16]. Du et al. characterize tightness measure-theoretically [5]. We give a finite-depth statement and an explicit non-tight model.

Attention and inference systems. Online softmax [19] underlies FlashAttention [3] and its successors, and KV caching is standard in serving systems [23, 13]. Our contribution is the algebraic packaging, a commutative monoid with a free-monoid homomorphism, and machine-checked proofs over arbitrary exponential characters. Permutation equivariance of attention without positional encodings is known [32].

Masked diffusion. The time-independence of the optimal denoiser was observed by Ou et al. [22], in the setting of D3PM [1], MDLM [26] and related models [28, 21]. We machine-check it, together with the any-order chain rule and the inexactness of parallel unmasking.

15 Limitations

Analysis on the reals enters only through exponential characters, and there is no measure theory, so sampling and tightness are stated through finite quantities. The oracle's temperature is restricted to $1/n$, which keeps softmax rational. The theorems hold for every temperature, but the oracle cannot evaluate an irrational softmax. Speculative decoding is proved for one drafted token. The sequence version follows by induction and is not yet formalized. Nothing is proved about floating-point arithmetic: implementations are compared with exact values within a stated tolerance. A decision that lies exactly on a floating-point boundary could in principle diverge, and none occurred in our runs. The diffusion theory covers absorbing diffusion, not Gaussian diffusion.

The chat protocol models tool calling in its common request-and-result form, not any vendor's specification. The measurements cover tokenizers and samplers; attention kernels, speculative decoders, chat renderers and diffusion samplers are evaluated by the oracle but not yet compared against implementations.

16 Conclusion

The inference side of LLMs is small enough to be given a precise meaning and large enough that its implementations disagree. A machine-checked semantics separates what is meaning from what is convention. It proves which reorderings, cachings and tilings are harmless, and it shows that some properties folklore expects, such as symmetric exact top-k, compositional compressing tokenizers and unambiguous plain-text templates, cannot exist. Connected to an exact oracle, the same definitions become a test instrument. Every divergence it surfaces is either a defect in an implementation or a convention that belongs in the semantics.

Availability. The Lean development, oracle, harness, and recorded measurement log accompany this paper.

Acknowledgments. The structure of this work follows `ledger-semantics` by John Wiegley. The formalization and harness were developed with the assistance of an AI system (Claude, Anthropic).

References

[1] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, 2021.

[5] L. Du, L. Torroba Hennigen, T. Pimentel, C. Meister, J. Eisner, and R. Cotterell. A measure-theoretic characterization of tight language models. In *ACL*, 2023.

[6] C. Elliott. Denotational design with type class morphisms. Technical Report 2009-01, LambdaPix, 2009.

[7] A. Fan, M. Lewis, and Y. Dauphin. Hierarchical neural story generation. In *ACL*, 2018.

[8] T. Fritz. A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics*, 370, 2020.

[9] J. L. Gastaldi, J. Terilla, L. Malagutti, B. DuSell, T. Vieira, and R. Cotterell. The foundations of tokenization: Statistical and computational concerns. arXiv:2407.11606, 2024.

[10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec*, 2023.

[11] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. In *ICLR*, 2020.

[12] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *CAV*, 2017.

[13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[14] J. Wiegley. ledger-semantics: Mathematical semantics of Ledger, the double-entry accounting tool. `https://github.com/ledger/ledger-semantics`, 2026.

[15] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[16] T. Lindvall. *Lectures on the Coupling Method*. Wiley, 1992.

[17] H. Liu, M. Zaharia, and P. Abbeel. Ring attention with blockwise transformers for near-infinite context. arXiv:2310.01889, 2023.

[18] The mathlib Community. The Lean mathematical library. In *CPP*, 2020.

[19] M. Milakov and N. Gimelshein. Online normalizer calculation for softmax. arXiv:1805.02867, 2018.

[20] M. N. Nguyen, A. Baker, C. Neo, A. Roush, A. Kirsch, and R. Shwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent LLM outputs. In *ICLR*, 2025.

[21] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li. Large language diffusion models. arXiv:2502.09992, 2025.

[22] J. Ou, S. Nie, K. Xue, F. Zhu, J. Sun, Z. Li, and C. Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *ICLR*, 2025.

[23] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. In *MLSys*, 2023.

[24] M. N. Rabe and C. Staats. Self-attention does not need $O(n^2)$ memory. arXiv:2112.05682, 2021.

[25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.

[26] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.

[27] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016.

[28] J. Shi, K. Han, Z. Wang, A. Doucet, and M. K. Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.

[29] T. Vieira, B. LeBrun, M. Giulianelli, J. L. Gastaldi, B. DuSell, J. Terilla, T. J. O'Donnell, and R. Cotterell. From language models over tokens to language models over characters. arXiv:2412.03719, 2024.

[30] T. Wolf et al. Transformers: State-of-the-art natural language processing. In *EMNLP: System Demonstrations*, 2020.

[31] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *ICLR*, 2024.

[32] C. Yun, S. Bhojanapalli, A. S. Rawat, S. J. Reddi, and S. Kumar. Are transformers universal approximators of sequence-to-sequence functions? In *ICLR*, 2020.

[33] V. Zouhar, C. Meister, J. L. Gastaldi, L. Du, T. Vieira, M. Sachan, and R. Cotterell. A formal perspective on byte-pair encoding. In *Findings of ACL*, 2023.